Privilege Sensor on Kubernetes: Our Environment

Oct 2, 2026 · @Curtis Muir

Overview

This is the record of our own Kubernetes sensor setup: the values we use, how the repo installs it, and what is still unverified. For setup steps, how the sensor works and generic troubleshooting, use *Privilege Sensor on Kubernetes: Guide*.

State on 2026-10-01: not measured. The sensor is installed only on the local OrbStack cluster. That cluster's API was not reachable when this was written (connection refused on its port, no Kubernetes containers running), so nothing here was checked against a live cluster. Earlier the same day every deployment in `ai-demo` had been scaled to 0 and the sensor daemonset was left in place.

Our values

All of these are set in `k8s/deploy.sh`, and each can be overridden by exporting its environment variable before running the script.

Setting	Our value	Environment variable
Namespace	`ai-demo`	fixed
Release name	`procyon-nodesensor`	fixed
Chart version	`0.1.11`	`PROCYON_NODESENSOR_CHART_VERSION`
`clusterID`	`k8-orbstack`	`PROCYON_NODESENSOR_CLUSTER_ID`
`serviceAccount.name`	`privilege-sensor`	`PROCYON_NODESENSOR_SERVICE_ACCOUNT`
`auth.audience`	`priv-nhi-ov4dgloai7iwbgu4heeonwn2763trc3x`	`PROCYON_NODESENSOR_AUDIENCE`
`controller.endpoint`	`hydra.privilege.pingone.com:443`	`PROCYON_NODESENSOR_CONTROLLER_ENDPOINT`
`controller.enabled`	`true`	fixed

The registered subject is therefore `system:serviceaccount:ai-demo:privilege-sensor`. The audience is an identifier, not a secret. It is a different identity from the Docker sensor's, which has its own subject and audience.

The daemonset is `procyon-nodesensor-procyon-nodesensor-chart` in `ai-demo`.

How it is installed here

The install is the function `install_procyon_nodesensor` in `k8s/deploy.sh`. It runs `helm upgrade --install`, then waits up to 180 seconds for the daemonset rollout and stops the deploy if that fails.

- **It runs only on OrbStack.** The function returns without doing anything unless the namespace is `ai-demo` and the current kubectl context is `orbstack`. A deploy to any other cluster installs no sensor.
- **It runs on every full deploy.** It is called at the end of a full `./k8s/deploy.sh`.
- **Skip it for one deploy** with `PROCYON_NODESENSOR_ENABLED=0 ./k8s/deploy.sh`.
- **It needs Helm** on the machine running the deploy; the script stops if `helm` is missing.
- **It passes no logging flags.** To turn on `flags.reportAll` or `flags.json`, add them by hand or edit the function.

The sensor does not start a model. The full deploy warms `llama-tier1` so there is something to detect; to generate findings, send a real inference request through an agent that uses the in-cluster LLM proxy.

Logs and dashboard

Alloy ships pod logs to Loki. `monitoring/alloy/config.k8s.alloy` tags the sensor's stream `component="privilege-sensor"`, matching on the container name `nodesensor`. The "Privilege Sensor" Grafana dashboard (`monitoring/grafana/dashboards/privilege-sensor.json`) queries that label, so the same dashboard serves the Docker and Kubernetes sensors.

Both were merged on 2026-10-01 (PR #512) and verified on Docker only. On Kubernetes the dashboards reach Grafana through the `grafana-dashboards` ConfigMap built by `k8s/create-secrets.sh`, so a cluster needs that script re-run to pick the new dashboard up.

Open questions on this machine

None of these was observed failing; each is something to check the next time the cluster is up.

- [ ] **How does Privilege get the OrbStack cluster's signing keys?** A local cluster's issuer is not reachable from Privilege, so the identity must hold a pasted JWKS or point at a published URL. The identity's configuration in the console was not checked.
- [ ] **Is there a second install?** Shell history shows sensor commands against a `privilege` namespace, with a release named for shadow AI. `k8s/deploy.sh` installs into `ai-demo` only. Run `helm list -A`.
- [ ] **Does the pod watch work on OrbStack?** `k8s/README.md` notes that pods cannot reach OrbStack's Kubernetes API. The sensor's pod attribution depends on that API; if it fails, findings will say `host` instead of a pod.
- [ ] **Is an enforcement policy applied?** The Docker sensor on this machine received an enforce-mode policy from the console. Check whether the Kubernetes identity has one too, and what its rules are.
- [ ] **Run the guide's verification steps** and replace the "not measured" note above with results.

Sources: `k8s/deploy.sh`, `k8s/README.md`, `k8s/create-secrets.sh`, `monitoring/alloy/config.k8s.alloy`.